A DEVOPS PROJECT REPORT SUBMITTED TO

THE NATIONAL INSTITUTE OF ENGINEERING, MYSURU

(An Autonomous Institute under Visvesvaraya Technological University, Belagavi)

in partial fulfillment for the award of degree of

Bachelor of Engineering
in
Computer Science and Engineering

Submitted By

Pallavi S E 4NI24CS421

Under the guidance of

Mr. Joyan Prajwal Alvares
Assistant Professor
Department of
CS&E
NIE, Mysuru

Department of Computer Science & Engineering

THE NATIONAL INSTITUTE OF ENGINEERING
(Autonomous Institution)
Mysuru - 570 008
2025-26

ABSTRACT

The DevOps laboratory experiments were conducted to understand the practical implementation of modern software development and deployment techniques using DevOps tools and technologies. The experiments include WSL installation, GitHub Classroom operations, Docker container management, Jenkins automation, and Docker-Jenkins integration.

The primary objective of these experiments is to gain hands-on experience in automation, version control, containerization, and Continuous Integration/Continuous Deployment (CI/CD) processes. GitHub is used for source code management and version control, Docker is used for creating and managing containers, and Jenkins is used to automate build and deployment processes.

Through these experiments, various DevOps concepts such as repository creation, Docker image building, container execution, Jenkins pipeline configuration, and automated deployment are implemented successfully. The experiments provide practical exposure to real-world DevOps workflows and help in understanding automation techniques used in software industries.

The overall work improves knowledge of CI/CD pipelines, reduces manual deployment effort, and demonstrates the importance of automation in modern software development environments.

ACKNOWLEDGMENT

I sincerely owe my gratitude to all the persons who helped and guided me in completing this DevOps project.

I am thankful to **Dr. Nagendra Parashar**, Principal, NIE College, Mysuru, for all the support she has rendered.

I thank **Dr. Anitha R**, Professor and Head, Department of Computer Science and Engineering, for her constant support and encouragement throughout the tenure for the project work.

I would like to sincerely thank my guide **Mr. Joyan Prajwal Alvares**, Assistant Professor, Department of Computer Science and Engineering, for providing relevant information, valuable guidance and encouragement to complete this project.

Pallavi S E 4NI24CS421

TABLE OF CONTENTS

CONTENTS

Chapter 1

WSL Installation

Introduction

Windows Subsystem for Linux (WSL) is a feature provided by Microsoft that enables users to run a Linux environment directly on the Windows operating system without the need for a separate virtual machine or dual boot setup.

WSL plays an important role in DevOps environments because many DevOps tools and platforms such as Docker, Git, Jenkins, Kubernetes, and shell scripting are designed primarily for Linux systems. By using WSL, developers can create a lightweight, efficient, and flexible development environment while continuing to work on Windows.

In this experiment, WSL and the Ubuntu Linux distribution were installed and configured successfully to create a Linux-based environment for performing various DevOps operations and automation tasks.

Purpose

The main purpose of this experiment is to create a Linux working environment on a Windows system for performing DevOps activities efficiently. The experiment helps students understand how Linux tools and commands can be integrated with Windows using WSL.

Objectives

The key objectives of this project are:

- To understand the concept and working of Windows Subsystem for Linux (WSL).
- To install and configure WSL on the Windows operating system.
- To install the Ubuntu Linux distribution in WSL.
- To learn and execute basic Linux terminal commands.
- To create a Linux-based environment for performing DevOps experiments and automation tasks.
- To gain practical knowledge of Linux environments used in modern software development and deployment processes.

System Requirements

Component	Requirement
Operating System	Windows 10 / Windows 11
RAM	Minimum 4 GB
Processor	Intel i3 / Intel i5 or above
Storage	Minimum 20 GB Free Space
Internet Connection	Required

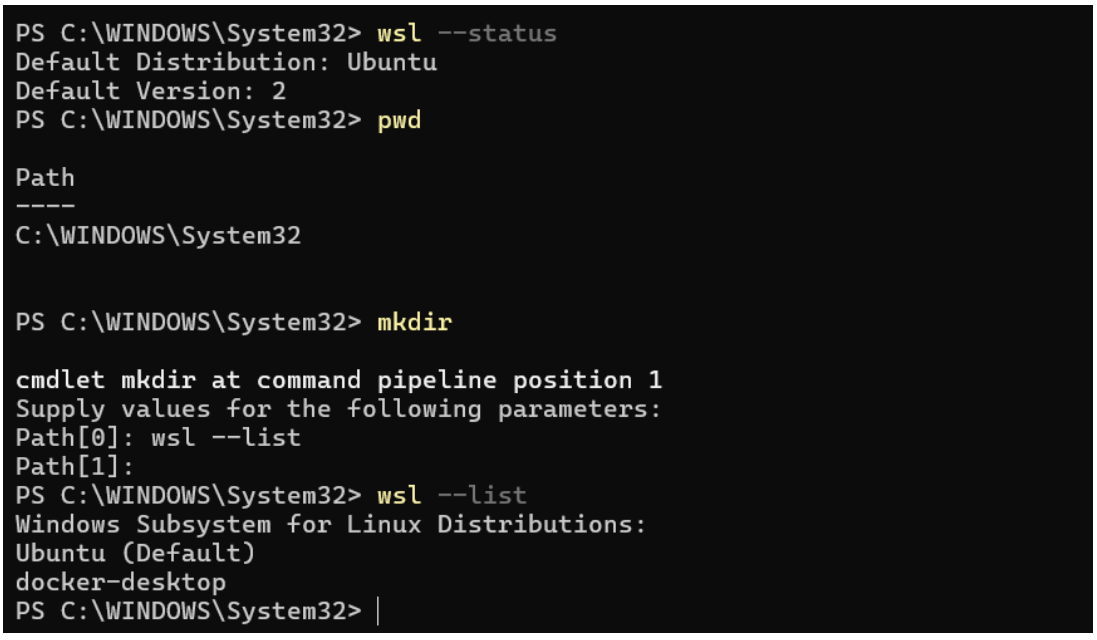
1.1 WSL Installation Steps

Step 1: Open PowerShell as Administrator

1. Click on the **Start Menu**.
2. Search for **PowerShell**.
3. Right-click on **Windows PowerShell** and select **Run as Administrator**

Step 2: Install WSL

Execute the following command in PowerShell: `wsl --install`



Step 3: Restart the System

After the installation is completed successfully, restart the computer to apply the changes.

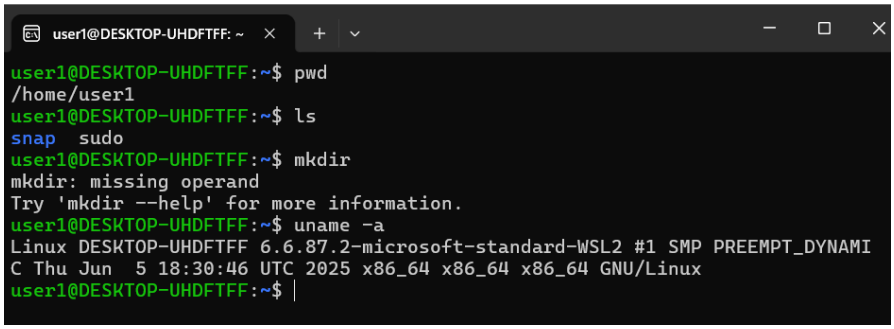
Step 4: Install Ubuntu

After restarting, Ubuntu installation starts automatically.

Wait until the installation process completes successfully.

Step 5: Create Username and Password

Set Linux username and password for Ubuntu environment.



1.2 Important Commands Used

Command	Description
wsl --install	Install WSL and Ubuntu
wsl --status	Show WSL installation status
wsl --list --verbose	Display installed Linux distributions
ubuntu	Launch Ubuntu terminal
pwd	Display current working directory
ls	List files and directories
mkdir foldername	Create a new directory
cd foldername	Change directory
clear	Clear terminal screen
touch filename.txt	Create a new file
cat filename.txt	Display file contents

Result:

Windows Subsystem for Linux (WSL) and Ubuntu Linux were installed and configured successfully on the Windows operating system. Basic Linux commands were executed successfully, and the Linux environment was prepared for performing DevOps experiments and automation tasks.

Chapter 2

GITHUB CLASSROOM EXPERIMENT

Introduction

GitHub is a popular web-based platform used for version control and source code management. It helps developers store, manage, and track changes in project files using Git. GitHub is widely used for collaborative software development and project sharing. GitHub Classroom provides students with an easy way to manage coding assignments and repositories. In this experiment, GitHub account creation, repository management, cloning, committing, and pushing code to GitHub were performed successfully.

Objectives

- To understand the basics of Git and GitHub.
- To create and manage GitHub repositories.
- To learn Git commands used for version control.
- To clone repositories from GitHub.
- To commit and push source code changes to GitHub.

GitHub Account Creation

Step 1:

Open a web browser and visit:

https://github.com

Step 2:

Click on the **"Sign Up"** button.

Step 3:

Enter the required details:

- Username
- Email ID
- Password

Step 4:

Verify the account and complete the registration process successfully

2.1 Repository Creation

1.Login to your GitHub account.

Click on the **"New Repository"** option.

Enter the repository name.

Choose the repository visibility:

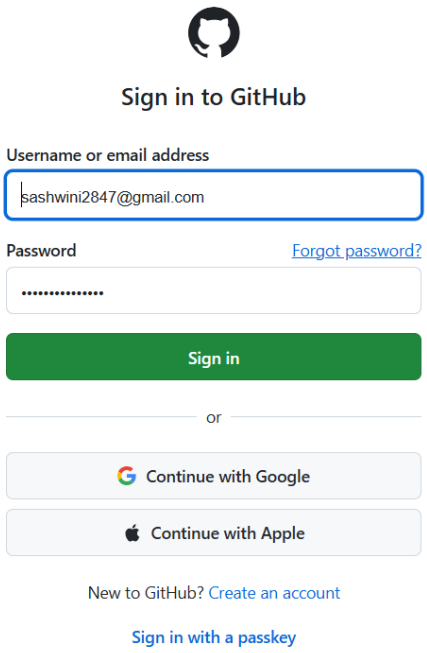
- Public
- Private

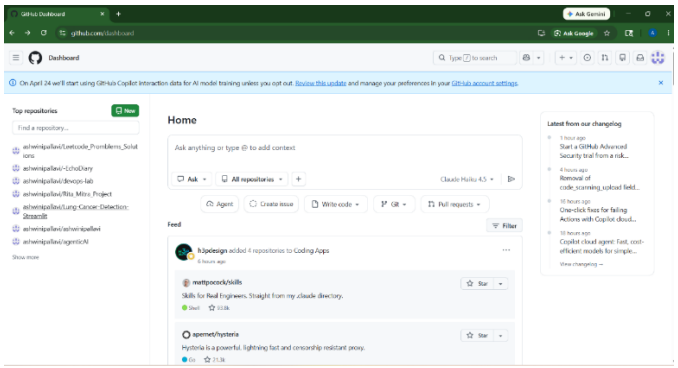
Step 3:

Enter the required details:

- Username
- Email ID
- Password

Click on "Create Repository" to create the repository successfully.





Git Installation and Configuration

Install Git using the following command: sudo apt install git -y

Check Git version: git --version

Configure username: git config --global user.name "username"

Configure username: git config --global user.name "username"

Configure email: git config --global user email- email@example.com

```
For more examples and ideas, visit:
 https://docs.docker.com/get-started/

root@DESKTOP-UHDFTFF:~# git config --global user.name  "Your Full Name"
root@DESKTOP-UHDFTFF:~# git config --global user.name  "ashwinipallavi"
root@DESKTOP-UHDFTFF:~# git config --global user.name  ""
root@DESKTOP-UHDFTFF:~# git config --global user.name

root@DESKTOP-UHDFTFF:~# git config --global user.email "sashwini2847@gmail.c
om"
root@DESKTOP-UHDFTFF:~# git config --global core.editor nano
root@DESKTOP-UHDFTFF:~# git config --global init.defaultBranch main
root@DESKTOP-UHDFTFF:~# git config --list
user.name=
user.email=sashwini2847@gmail.com
core.editor=nano
init.defaultbranch=main
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
remote.origin.url=https://github.com/joyanprajwalalvares-max/git--class-D3.g
it
remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
```

Cloning Repository:

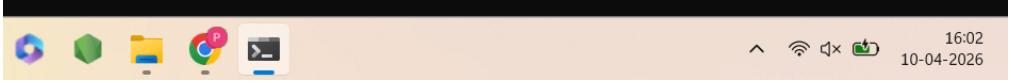
Clone the GitHub repository using the following command

- git clone https://github.com/Sneha14-c/devops06.git

Move into repository folder:

- cd devops06

```
root@DESKTOP-UHDFTFF:~/devops-lab# git remote -v
origin  https://github.com/ashwinipallavi/devops-lab.git (fetch)
origin  https://github.com/ashwinipallavi/devops-lab.git (push)
root@DESKTOP-UHDFTFF:~/devops-lab# git remote set-url origin git@github.com:
ashwinipallavi/devops-lab.git
root@DESKTOP-UHDFTFF:~/devops-lab# git remote -v
origin  git@github.com:ashwinipallavi/devops-lab.git (fetch)
origin  git@github.com:ashwinipallavi/devops-lab.git (push)
root@DESKTOP-UHDFTFF:~/devops-lab#
root@DESKTOP-UHDFTFF:~/devops-lab# eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
Agent pid 1460
Identity added: /root/.ssh/id_ed25519 (sashwini2847@gmail.com)
root@DESKTOP-UHDFTFF:~/devops-lab# ssh-add -l
256 SHA256:aB4EIrtLUehVZn8LFKMbRVguhPJUKrBHd6g54Kc0kgI sashwini2847@gmail.co
m (ED25519)
root@DESKTOP-UHDFTFF:~/devops-lab# ssh -T git@github.com
```

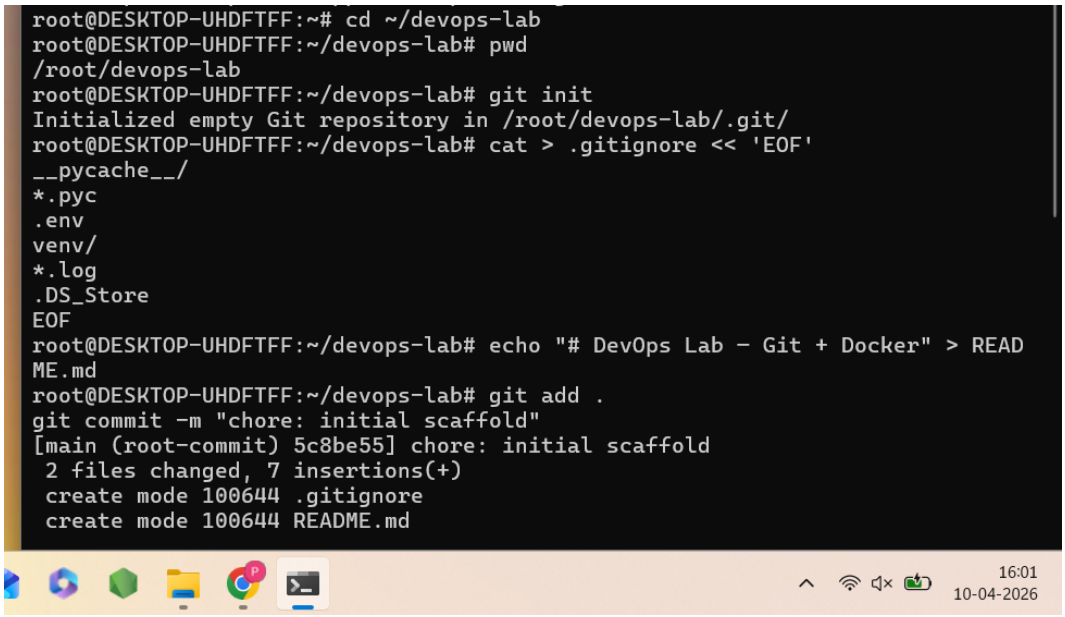


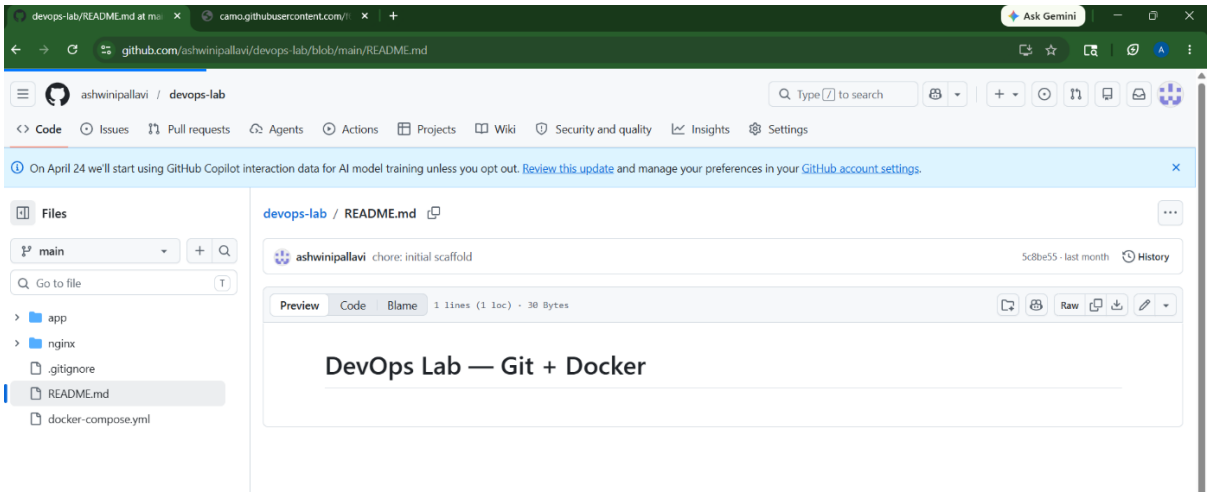
2.2 Pushing Code to GitHub

Add files: git add .

Commit changes: git commit -m "Initial Commit"

Push files: git push origin main





Result

GitHub account and repository were created successfully. Git commands such as clone, add, commit, and push were executed successfully, and source code files were uploaded to the GitHub repository.

Chapter 3

DOCKERS 1

Introduction

Docker is an open-source containerization platform used to develop, deploy, and run applications inside lightweight containers. Docker containers package the application along with all required dependencies, libraries, and configurations, ensuring consistent execution across different environments. Docker helps developers simplify application deployment and improves resource utilization compared to traditional virtual machines. It is widely used in DevOps environments for automation, scalability, and efficient software deployment.

Objectives

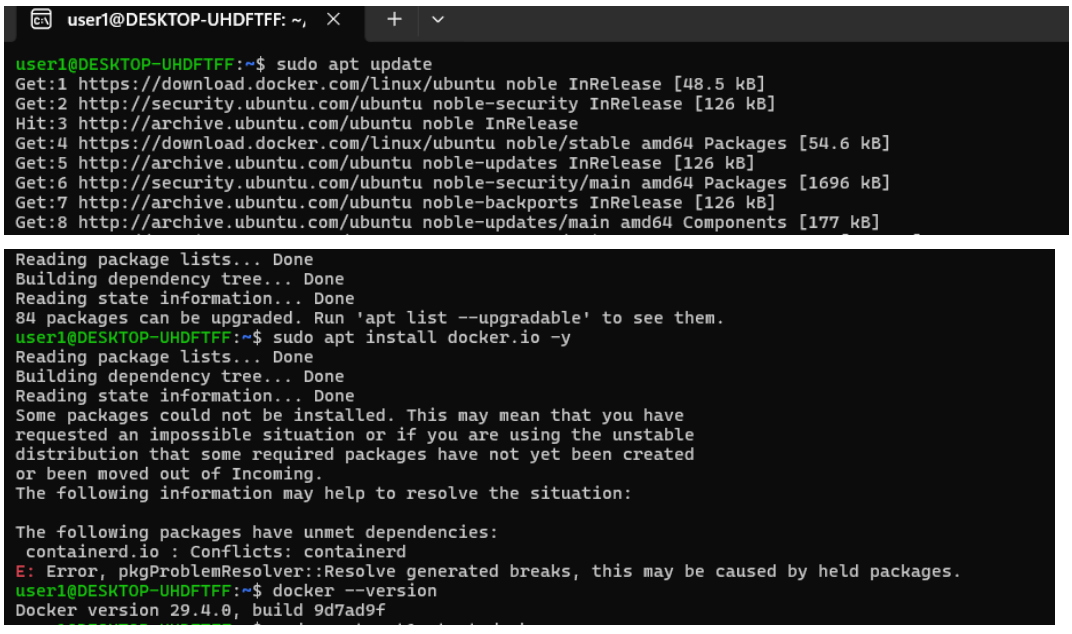
- To understand the basics of Docker.
- To install Docker on Ubuntu/Linux.
- To learn Docker commands and container operations.
- To pull Docker images from Docker Hub.
- To create and run Docker containers.

3.1 Docker Installation

Step 1: Update system packages: sudo apt update

Step 2: Install Docker: sudo apt install docker.io -y

Step 3: Verify Docker installation: docker –version

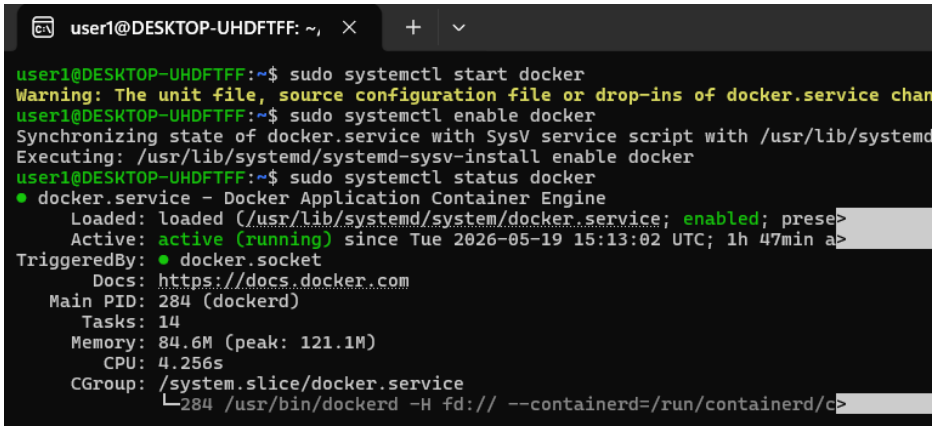


Starting Docker Service

Start Docker service: sudo systemctl start docker

Enable Docker service: sudo systemctl enable docker

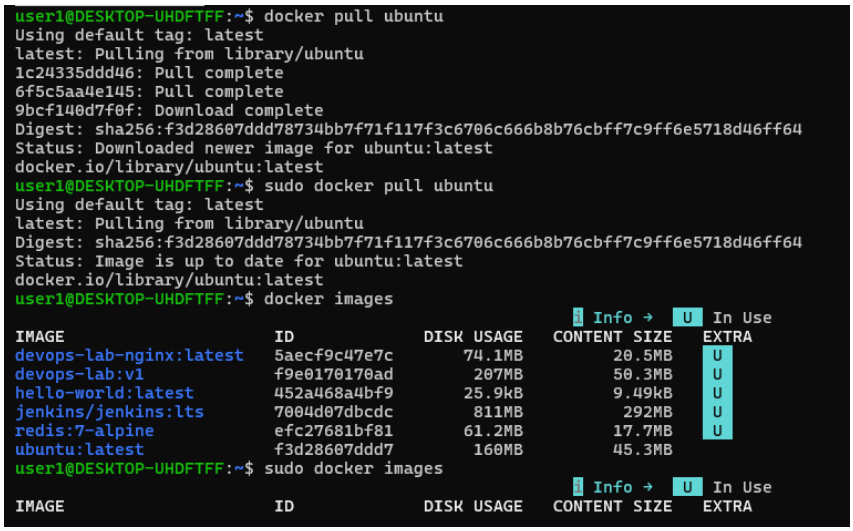
Check Docker status: sudo systemctl status docker



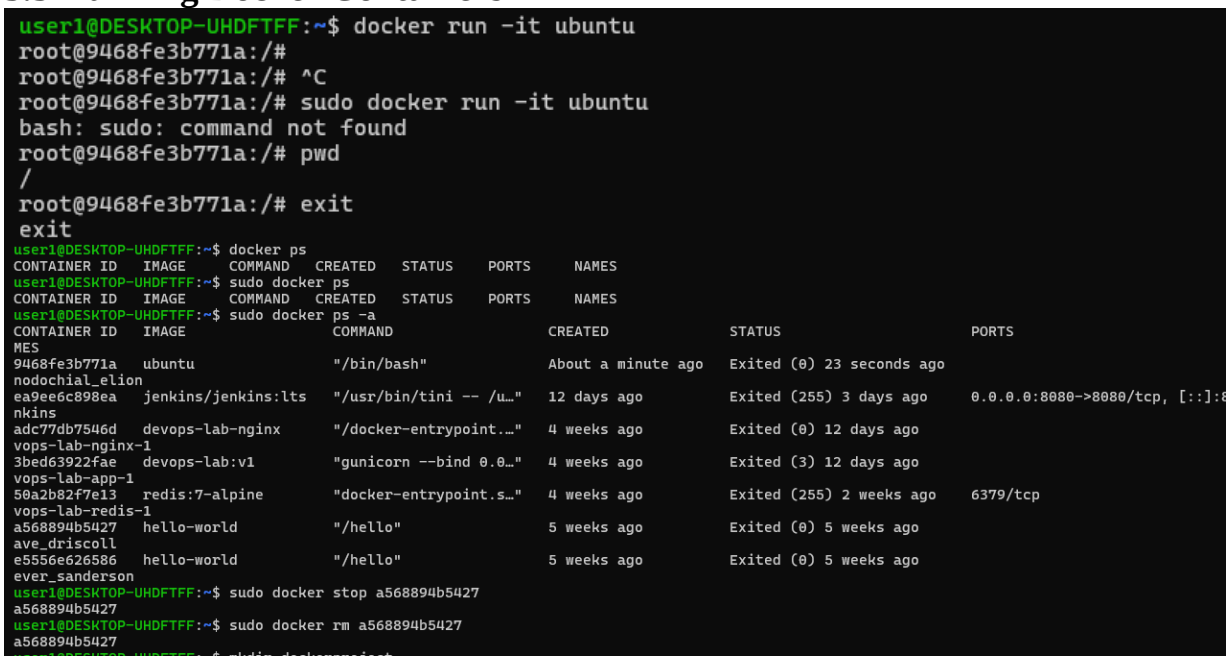
3.2 Pulling Docker Images

Download Ubuntu image from Docker Hub: docker pull ubuntu

Check downloaded images: docker images



3.3 Running Docker Containers



Result

Docker was installed and configured successfully on Ubuntu/Linux. Docker images were downloaded successfully, and containers were created and executed using basic Docker commands.

Chapter 4

DOCKERS 2

Introduction

Docker allows applications to run inside isolated containers with all required dependencies. Docker images are used to create containers, and Dockerfiles help automate the image creation process. A Dockerfile is a text file that contains instructions for building Docker images. Using Dockerfiles, developers can automate application deployment and maintain consistency across different systems. In this experiment, Dockerfile creation, Docker image building, container execution, port mapping, and container management operations are performed successfully.

Objectives

- To understand Docker image creation.
- To create and use Dockerfiles.
- To build Docker images using Dockerfile.
- To run web applications inside Docker containers.
- To learn Docker container management operations.

4.1 Creating Dockerfile

Step 1: Create project directory: mkdir dockerproject

Step 2: Move into project directory: cd dockerproject

Step 3: Create HTML file: nano index.html

Sample HTML code:

```
<!DOCTYPE html>
<html>
<head>
  <title>Docker Project</title>
</head>
<body>
  <h1>Docker Container Running Successfully</h1>
</body>
</html>
```

Step 4: Create Dockerfile: nano Dockerfile

Dockerfile content:

FROM nginx:latest

COPY index.html /usr/share/nginx/html/index.html

```
user1@DESKTOP-UHDFTFF:~$ mkdir dockerproject
user1@DESKTOP-UHDFTFF:~$ cd dockerproject
user1@DESKTOP-UHDFTFF:~/dockerproject$ nano index.html
user1@DESKTOP-UHDFTFF:~/dockerproject$ nano Dockerfile
```

4.2 Building Docker Image

Build Docker image using: docker build -t mydockerapp .

Check Docker images: docker images

```
[sudo] password for sowjanya:
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  3.072kB
Step 1/2 : FROM nginx:latest
latest: Pulling from library/nginx
57fb71246055: Pulling fs layer
834a05acfff4: Pulling fs layer
60ad0c2ccfc6: Pulling fs layer
735e1c628373: Pulling fs layer
9f7bde970101: Pulling fs layer
1bb34ee717a4: Pulling fs layer
030365c1a354: Pulling fs layer
89fc4d629470: Download complete
9f7bde970101: Download complete
030365c1a354: Download complete
1bb34ee717a4: Download complete
735e1c628373: Download complete
834a05acfff4: Download complete
73d0e4d315b3: Download complete
60ad0c2ccfc6: Download complete
57fb71246055: Download complete
57fb71246055: Pull complete
834a05acfff4: Pull complete
60ad0c2ccfc6: Pull complete
1bb34ee717a4: Pull complete
9f7bde970101: Pull complete
030365c1a354: Pull complete
735e1c628373: Pull complete
Digest: sha256:06aa3d7be10bc6307990c81bdca075793132e9163391abc370c015e344e23128
Status: Downloaded newer image for nginx:latest
 ---> 06aa3d7be10b
Step 2/2 : COPY index.html /usr/share/nginx/html/index.html
 ---> 7d865aa22550
Successfully built 7d865aa22550
Successfully tagged mydockerapp:latest
```

Running Docker Container

Run Docker container: docker run -d -p 8080:80 mydockerapp

Check running containers: docker ps

4.3 Port Mapping

Port mapping connects container ports to system ports.

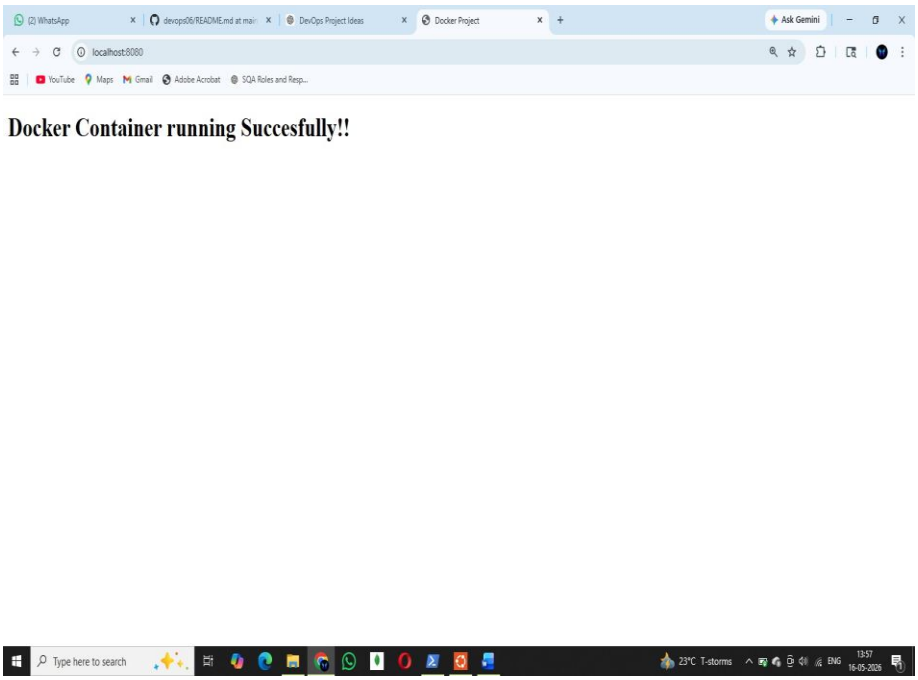
Example: docker run -d -p 8080:80 mydockerapp

Here:

- 8080 → System port
- 80 → Container port

The application can be accessed using:

http://localhost:8080



Docker Container Management

```
user1@DESKTOP-UHDFTFF:~$ sudo docker stop a568894b5427
a568894b5427
user1@DESKTOP-UHDFTFF:~$ sudo docker rm a568894b5427
a568894b5427
```

Docker Commands Used

Command	Description
docker build	Build a Docker image
docker run	Run a Docker container
docker ps	Display running containers
docker stop	Stop a running container
docker start	Start a stopped container
docker rm	Remove a container

Result

The Dockerfile was created successfully, and the Docker image was built using the Dockerfile. The Docker container was executed successfully, and the web application was deployed and accessed through the browser using port mapping.

Chapter 5

JENKINS

Introduction

Jenkins is an open-source automation server used for Continuous Integration and Continuous Deployment (CI/CD). Jenkins automates software development tasks such as building, testing, and deploying applications. It helps developers reduce manual work and improve software delivery speed and reliability. Jenkins supports pipelines and plugins that allow integration with various DevOps tools such as Docker and GitHub. It continuously monitors source code repositories and automatically performs build and deployment operations whenever code changes are detected. In this experiment, Jenkins installation, configuration, service management, and pipeline job creation are performed successfully.

Objectives

- To understand the basics of Jenkins.
- To install Jenkins on Ubuntu/Linux.
- To configure Jenkins server.
- To create Jenkins jobs and pipelines.
- To automate software build and deployment tasks.

Java Installation

Jenkins requires Java for execution.

Update system packages: sudo apt update

Install Java: sudo apt install openjdk-17-jdk -y

Verify Java installation: java -version

```
PS C:\WINDOWS\System32> java --version
java 17.0.18 2026-01-20 LTS
Java(TM) SE Runtime Environment (build 17.0.18+8-LTS-264)
Java HotSpot(TM) 64-Bit Server VM (build 17.0.18+8-LTS-264, mixed mode, shar
ing)
PS C:\WINDOWS\System32>
```

Jenkins Installation

Step 1: Download Jenkins key: sudo wget -O /usr/share/keyrings/jenkins-keyring.asc\ https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key

Step 2: Add Jenkins repository: echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" \ https://pkg.jenkins.io/debian-stable binary/ | sudo tee \ /etc/apt/sources.list.d/jenkins.list > /dev/null

Step 3: Install Jenkins: sudo apt update sudo apt install jenkins

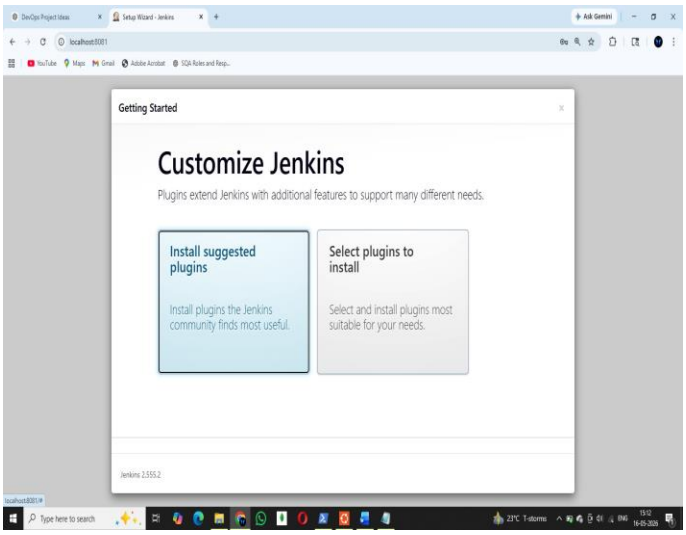
5.1 Jenkins Configuration

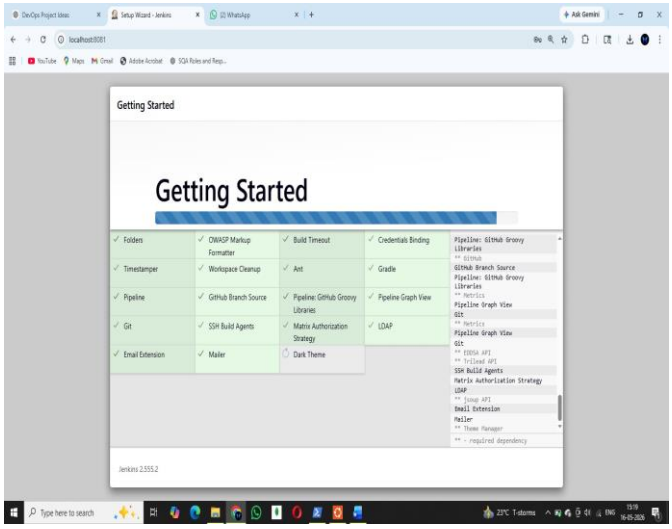
Step 1: Install suggested plugins.

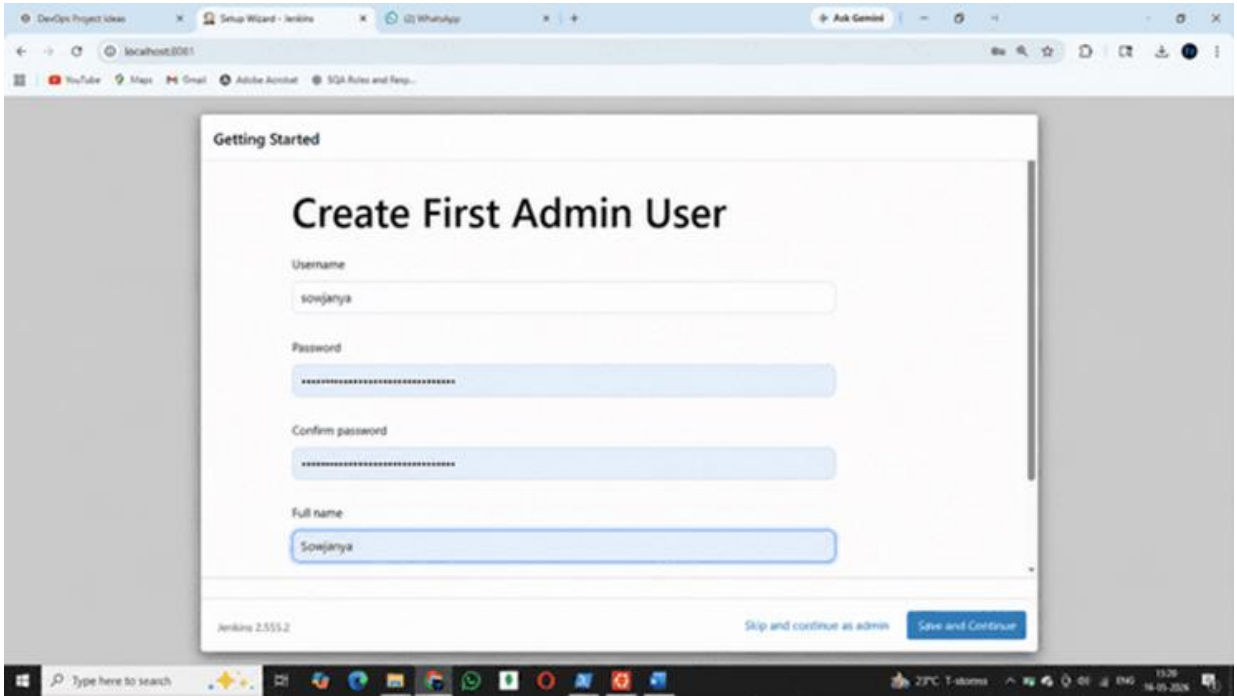
Step 2: Create Jenkins administrator account.

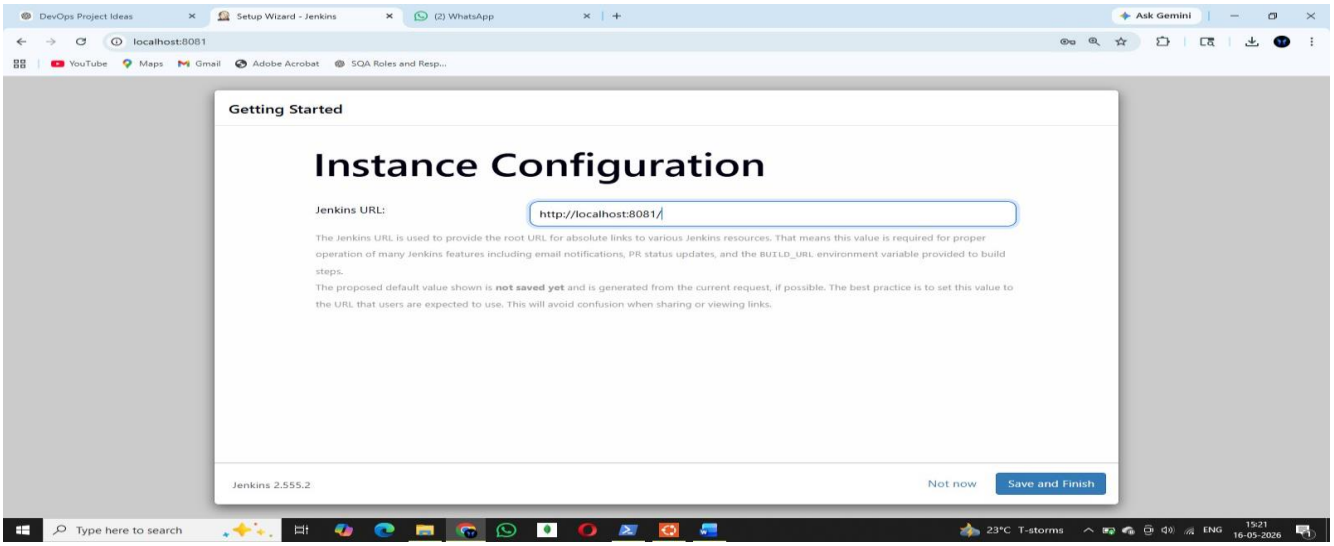
Step 3: Configure Jenkins URL.

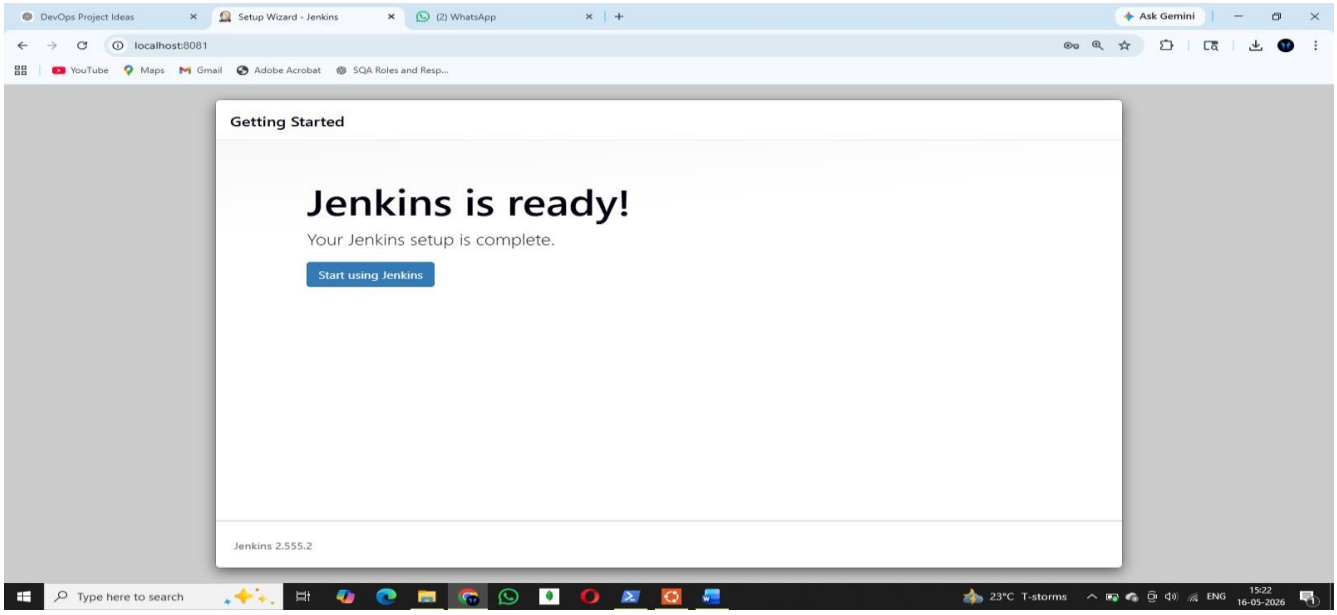
Step 4: Login to Jenkins dashboard.

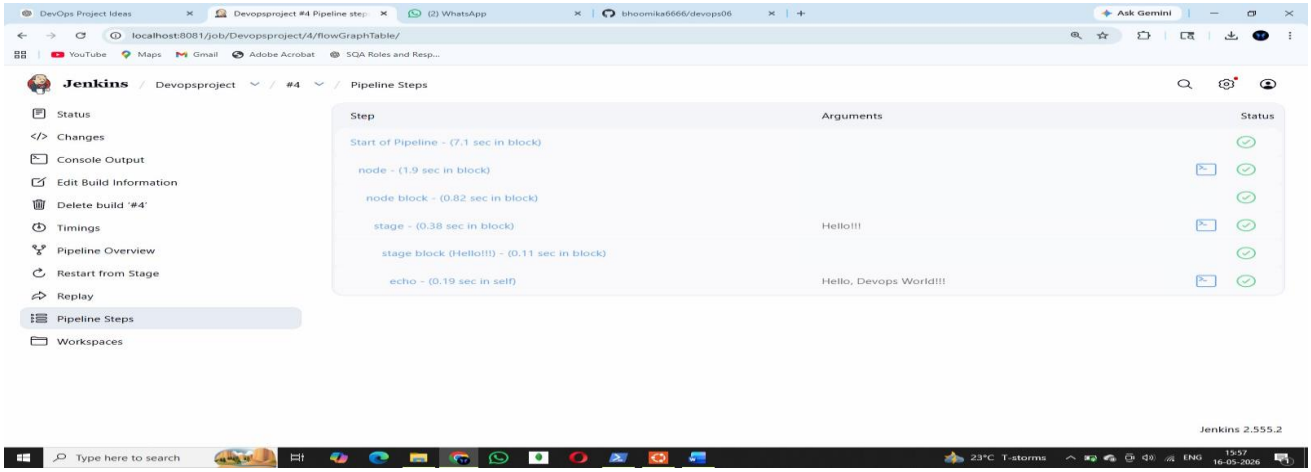







Getting Started
Instance Configuration
Jenkins URL:
http://localhost:8081/
The Jenkins URL is used to provide the root URL for absolute links to various Jenkins resources. That means this value is required for proper operation of many Jenkins features including email notifications, PR status updates, and the BUILD_URL environment variable provided to build steps.
The proposed default value shown is not saved yet and is generated from the current request, if possible. The best practice is to set this value to the URL that users are expected to use. This will avoid confusion when sharing or viewing links.
Jenkins 2.555.2
Not now
Save and Finish


Getting Started
Jenkins is ready!
Your Jenkins setup is complete.
Start using Jenkins
Jenkins 2.555.2


Jenkins
Devopsproject
#4
Pipeline Steps
Status
Changes
Console Output
Edit Build Information
Delete build '#4'
Timings
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces
Step
Arguments
Status
Start of Pipeline - (7.1 sec in block)
node - (1.9 sec in block)
node block - (0.82 sec in block)
stage - (0.38 sec in block)
Hello!!!
stage block (Hello!!!) - (0.11 sec in block)
echo - (0.19 sec in self)
Hello, Devops World!!!
Jenkins 2.555.2

Important Commands

Command	Description
java -version	Verify Java installation
sudo systemctl start jenkins	Start Jenkins service
sudo systemctl enable jenkins	Enable Jenkins service
sudo systemctl status jenkins	Check Jenkins service status
sudo cat /var/lib/jenkins/secrets/initialAdminPassword	Display Jenkins administrator password

Result:

Jenkins was installed and configured successfully on Ubuntu/Linux. The Jenkins dashboard was accessed successfully through the web browser, and pipeline jobs were created to automate software build and deployment processes. The experiment helped in understanding Continuous Integration and Continuous Deployment (CI/CD) concepts using Jenkins.

Chapter 6

DOCKER AND KUBERNETES

Aim

To deploy Docker containers using Kubernetes orchestration platform.

Theory

Kubernetes is a container orchestration platform used to automate deployment, scaling, and management of containerized applications.Kubernetes manages multiple containers efficiently and improves application scalability and reliability.

Commands Used

kubectl version

kubectl create deployment myapp --image=nginx

kubectl get pods

kubectl get services

kubectl expose deployment myapp --type=NodePort --port=8

Procedure

Install Kubernetes tools.

Create Kubernetes deployment.

Verify running pods.

Expose deployment using service.

Access deployed application.

Output

Docker containers were successfully deployed and managed using Kubernetes.

Result

Kubernetes deployment and service management were successfully implemented.

```
root@DESKTOP-UHDFTFF:~/devops-lab# minikube start --driver=docker
😄  minikube v1.38.1 on Ubuntu 24.04 (kvm/amd64)
✨  Using the docker driver based on user configuration
🛑  The "docker" driver should not be used with root privileges. If you wish to continue as root, use --force.
💡  If you are running minikube within a VM, consider using --driver=none:
📘    https://minikube.sigs.k8s.io/docs/reference/drivers/none/
```

```
root@DESKTOP-UHDFTFF:~/devops-lab# kubectl version
Client Version: v1.35.5
Kustomize Version: v5.7.1
The connection to the server localhost:8080 was refused - did you specify the right host or port?
```

Minikube installation

```
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# docker ps
CONTAINER ID   IMAGE                    COMMAND                  CREATED        STATUS                            PORTS                                       NAMES
291cba4b3b00   snake-game:1.0           "dumb-init -- node a…"   11 seconds ago Up 11 seconds (health: starting)  0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp  snake
-game-test
c1589d1d88cd   devops-lab-nginx         "/docker-entrypoint.…"   13 days ago    Up 15 minutes                     0.0.0.0:80->80/tcp, [::]:80->80/tcp         devop
s-lab-nginx-1
d7f74a59b836   devops-lab:v2-multistage "gunicorn --bind 0.0…"   13 days ago    Up 15 minutes (healthy)           5000/tcp                                    devop
s-lab-app-1
```

Snake Game container running

```
draw();^CextDirection = moves[e.key];e => {0),) {art.y === head.y)
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# npm start

> snake-game@1.0.0 start
> node app.js

 🐍  Snake Game Server running on port 3000
 🌐  Health: http://localhost:3000/api/health
^C
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# ^C
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# dir public
game.js  index.html  style.css
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# npm start

> snake-game@1.0.0 start
> node app.js

 🐍  Snake Game Server running on port 3000
 🌐  Health: http://localhost:3000/api/health
```

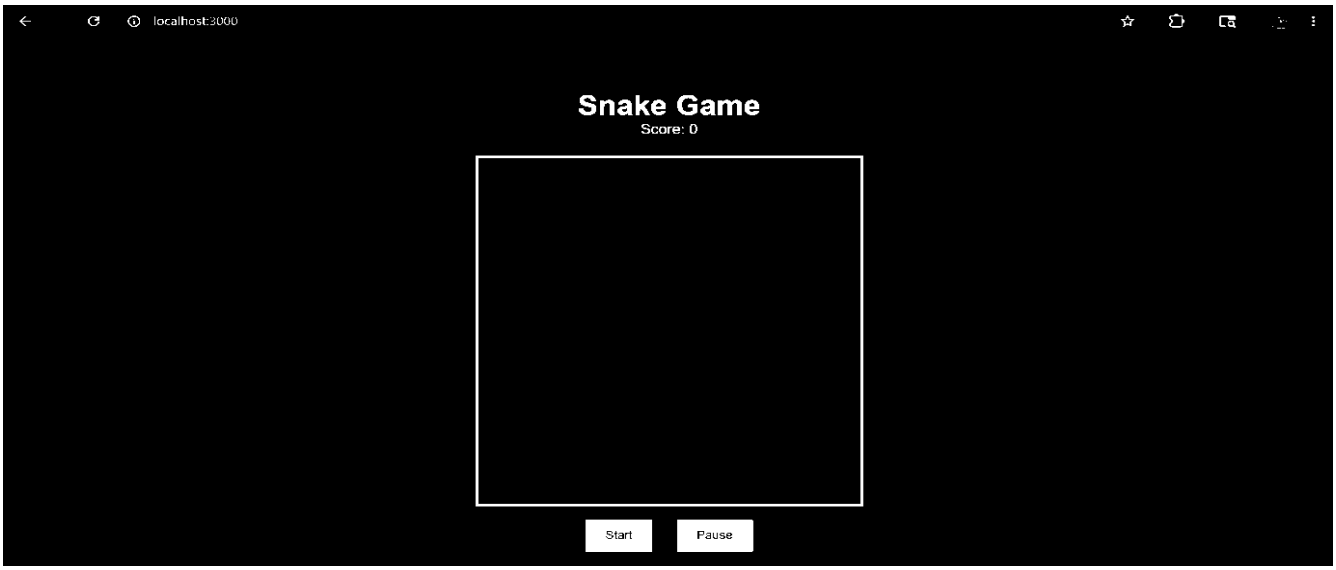
Running Snake game

```
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# minikube service snake-game-service --url
http://192.168.49.2:30782
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl scale deployment snake-game-deployment --replicas=5
deployment.apps/snake-game-deployment scaled
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# docker tag snake-game:1.0 snake-game:2.0
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl set image deployment/snake-game-deployment snake-game=snake-game:2.0
deployment.apps/snake-game-deployment image updated
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl rollout undo deployment/snake-game-deployment
deployment.apps/snake-game-deployment rolled back
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl delete -f k8s/
deployment.apps "snake-game-deployment" deleted from default namespace
service "snake-game-service" deleted from default namespace
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# minikube stop
✋  Stopping node "minikube"  ...
🛑  Powering off "minikube" via SSH ...
🛑  1 node stopped.
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab#
```

Scaling k8s and stopping minikube

```
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl apply -f k8s/deployment.yaml
deployment.apps/snake-game-deployment created
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl apply -f k8s/service.yaml
service/snake-game-service created
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl get pods
NAME                                   READY   STATUS    RESTARTS   AGE
snake-game-deployment-5669bddcc4-gvndg   0/1     Running   0          11s
snake-game-deployment-5669bddcc4-jw4w8   0/1     Running   0          11s
snake-game-deployment-5669bddcc4-pj9nt   0/1     Running   0          11s
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl get deployments
NAME                   READY   UP-TO-DATE   AVAILABLE   AGE
snake-game-deployment   3/3     3            3           17s
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab# kubectl get services
NAME                 TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
kubernetes           ClusterIP      10.96.0.1       <none>        443/TCP        9m9s
snake-game-service   LoadBalancer   10.105.174.43   <pending>     80:30782/TCP   17s
root@DESKTOP-K6NKQ1A:~/docker-k8s-lab#
```

Service pods and deployment



Snake game in browser

Output

Docker containers were successfully deployed and managed using Kubernetes.

Result

Kubernetes deployment and service management were successfully implemented.

CONCLUSION

The DevOps laboratory experiments were completed successfully using various DevOps tools and technologies such as GitHub, Docker, and Jenkins. The experiments provided practical knowledge of Linux environments, version control systems, containerization, and Continuous Integration/Continuous Deployment (CI/CD) processes.

Initially, WSL and Ubuntu Linux were installed and configured successfully to create a Linux-based environment for DevOps operations. GitHub experiments helped in understanding repository management, source code version control, cloning repositories, and pushing code changes.

Docker experiments provided hands-on experience in creating Docker containers, building Docker images, writing Dockerfiles, and deploying applications inside containers. Jenkins experiments demonstrated the automation of software build and deployment tasks using CI/CD pipelines. Finally, Docker and Jenkins integration enabled automated application deployment whenever code changes were pushed to the GitHub repository. The implemented CI/CD pipeline reduced manual effort, improved deployment speed, and increased software reliability. Overall, these experiments helped in understanding real-world DevOps workflows, automation techniques, and modern software deployment practices used in the software industry.

REFERENCES

- Docker Documentation – https://docs.docker.com
- Jenkins Documentation – https://www.jenkins.io/doc/
- Kubernetes Documentation – https://kubernetes.io/docs/
- GitHub Documentation – https://docs.github.com
- Microsoft WSL Documentation – https://learn.microsoft.com/en-us/windows/wsl/
- Docker and Kubernetes Lab Guide
- Jenkins CI/CD Lab Guide
- Git WSL Lab Guide.